

Auto Shop Service Advisor and Intake Bot

Calvin A. Prepetit | CSC-128 Capstone | October 2026

1. Audience and problem

This bot is for customers who need help explaining a vehicle issue and preparing a service request. I continued the auto shop theme from my earlier assignments because it gave me a practical problem to build on. A customer should be able to say, "My car makes a weird rattling sound," without knowing the name of a part or which department should handle it.

My professor pointed out that customers may know very little about cars. I addressed that by collecting the customer's description and up to 3 focused observations before showing a technician note and suggested service areas. Questions cover onset and useful details such as sounds, conditions, or lights. Unknown answers and Skip question allow progress. Symptom dates and times stay in the issue history, separate from appointment choices. Suggestions are for inspection, not a diagnosis or a promise of a repair.

2. Intents and entities

Service issue triage takes the customer's description and useful follow-up details. Its result is a technician note and suggested departments with a short explanation. The note offers scheduling; yes accepts it and shows openings without booking. The customer can instead request an unscheduled ticket or summary. Describing work alone does not choose a ticket. The 5 approved departments are electrical, drivability, interior, exterior, and maintenance.

Appointment setting takes the collected concern, customer name, vehicle, and selected weekday/time. It checks the fictional schedule and produces a reviewed appointment request with an APT identifier. One intake slot can serve multiple departments. Appointment times are checked again at saving and reserved within that browser session.

Service request ticket creation produces an SR ticket from reviewed customer and issue details. A ticket can be created without an appointment, or include a selected available time. This makes it useful when the customer wants the issue documented before choosing a visit.

Technician intake summary with confirmation produces a short customer-reported service note alongside the original messages. It can be reviewed and confirmed without creating a ticket or reserving an appointment. The same summary review is included in the final previews for saved requests.

The main entities are the issue details, customer name, vehicle description, expected work when supplied, approved departments, weekday, and time. The conversation keeps information across turns and asks only for missing details. I kept each request to 1 vehicle; multiple vehicles can be entered as separate requests. Vehicle count was optional in the proposal, and this keeps the schedule and records easier to understand.

3. Deterministic code and model use

To reduce API usage, AI initially selects up to 3 useful questions for missing facts. Python collects their answers locally, then sends them together for one complete AI-written note. Enough initial detail skips the interview. Clear routine jobs, task commands, identity, confirmation and displayed time choices work locally. A weekday change clears the old time. Ambiguous replies still use AI; timed rate-limit messages and provider limits remain.

I reused ideas from Assignment 4 for slot filling and corrections, Assignment 6 for a small document set and retrieval, and Assignment 7 for preview-before-write confirmation and tool validation. The interface is in `app.py`. Conversation control, model interpretation, shop information, and record tools have separate files with specific purposes.

The language model interprets uneven wording and typos, suggests department matches, selects clarification, and drafts the note. It interprets ambiguous agreement and distinguishes acceptance of intake routing from a scheduling question. Its structured response proposes values; it does not write records. It receives current fields, issue history, observations, and the approved department guide, without repeatedly sending chat previews.

Python validates proposed fields against the customer's text. It normalizes weekdays and times, limits departments, checks availability, creates identifiers, and controls confirmation. A bare time must match a unique displayed opening; a weekday-only correction clears the old time. A small intake helper prioritizes explicit danger and unsupported decisions, preserves clearly requested tasks, and rejects symptoms mislabeled as requested repair work. These controls support the model rather than making it responsible for safety or saving.

The customer sees exact details before a write. Corrections invalidate the preview. Clear agreement, including 'yes that looks fine', works in Python; unfamiliar replies use the model. Routing agreement plus a visit question advances intake without saving. Questions, thanks alone, and conditional agreement do not authorize a write. Each preview has a revision and confirmation identifier to prevent duplicate or stale saves. Unspecified work is labeled diagnostic inspection, not an assumed repair.

Original messages come directly from conversation data, not model memory. The model writes concise, evidence-backed observations by topic: concern, onset, conditions, location, and other details. Python retains these across turns and joins them into a readable note without raw-answer appendices. Corrections replace a topic; supplied details suppress repeated questions. Starting concerns also have a factual fallback. Scheduling does not rewrite the note; availability comes before identity, which remains required before saving. Customers review and can download demo requests.

The fictional shop guide covers department descriptions, what to bring, drop-off instructions, hours, and demo request limitations. Retrieval uses TF-IDF and cosine similarity, adapted from Assignment 6. Policy answers show approved text with its source; an empty result leads to a human referral. Tools return availability, prepare previews, and save validated records. There are no payment, deletion, arbitrary execution, or real repair-authorization tools.

Storage is intentionally session-only. A refresh or lost session can clear records, and each visitor has a separate fictional schedule. This is not a shared shop booking database. Start Over clears the unfinished conversation but preserves saved requests in the current session. The app discloses these limits and that it is automated software. Messages are sent to Groq, so the demo asks customers to use fictional details.

4. Actual testing failures and changes

Customer testing found omitted observations and added work, invented times, unnecessary routine questions, and ignored corrections. All 122 no-key tests pass, including the 2-call interview and local scheduling. A compact prompt requests JSON with no automatic format retry. Useful intake questions override a contradictory empty issue flag; malformed optional work does not discard symptoms. Live testing caught an empty clarification list and a battery-uncertainty reply incorrectly removing a lighting sentence. I normalized empty clarification and restricted the lighting safeguard to actual corrections. Complete observations skip extra questions; routine jobs need no interview. Both tire and oil work remain in the note. Missing identity is requested before saving. The fixes are published; no record is saved automatically.

The first deployed shell asked several questions at once and routed concerns too early. Python now shows 1 question per turn, collects up to 3 observations, and provides Skip question. Symptom times no longer become appointment slots. The final live no-start rehearsal also checked follow-up batching and preserved facts when the model returned an empty note.

Live testing found a tire request with an empty note, causing repeated intake. I separated requested work from the symptom flag, added a factual fallback, and preserved explicitly chosen tasks. Tests check preview progression and saved values, not just extraction.

Live boundary tests found severe brake loss entering ordinary intake and warranty questions receiving unrelated demo policy. An insurance refusal was classified correctly but discarded by the Python decline branch. I added explicit boundary priority, kept refusal messages across action combinations, and checked policy-topic relevance before retrieval. Regression cases include negated danger and a model decline; the retest checks customer-visible referrals and confirms that nothing was saved.

Other fixes keep definition questions separate from actions, resolve explicit weekdays and displayed times locally, and replace typo-filled appendices with evidence-backed observations. Describing an oil change no longer chooses a ticket automatically. Requested work is not repair authorization, speed units are not guessed, and moving stalls receive a warning. Timed rate-limit replies preserve details. The outage simulation uses no API quota; provider availability cannot be guaranteed.

5. Refusal and human handoff

The bot refuses exact repair prices, warranty decisions, insurance claims, recall lookups, and definitive diagnoses, and refers customers to appropriate human help. Serious safety concerns receive an immediate safety and towing referral that remains visible. The customer can continue documenting the concern and prepare a fictional request for advisor review; this is not permission to drive or a real booking. Changes to saved records also require a service advisor; cancel clears unfinished work.

6. What I would build with another 4 weeks

I would add a shared persistent database and a staff view so reservations survive refreshes and different visitors cannot take the same real slot. I would replace the fictional documents with policies reviewed by an actual shop, add calendar dates, and let staff manage availability. I would also expand the live language tests, especially for vague descriptions, overlapping departments, and corrections that include several changes at once. A technician-reviewed summary should remain connected to the customer's original wording.